

RAG for Multi-turn Conversational AI Systems

1st Nguyễn Văn Thịnh
MSV: 23021726

3rd Ngô Viết Thuyết
MSV: 23021730

4th Nguyễn Trường Sơn
MSV: 23021686

2nd Mai Đức Văn
MSV: 23021746

5th Nguyễn Minh Phúc
MSV: 23021662

Index Terms—RAG, Conversational AI, Multi-turn context, LLM, Information Retrieval

I. GIỚI THIỆU

Sự bùng nổ của các mô hình ngôn ngữ lớn (LLMs) đã mở ra một kỷ nguyên mới cho trí tuệ nhân tạo. Tuy nhiên, sự phát triển ấy đi kèm với những nhược điểm cố hữu như kiến thức bị giới hạn tại thời điểm huấn luyện (knowledge cutoff) và hiện tượng bịa thông tin (hallucination). Để giải quyết vấn đề này, kiến trúc Retrieval-Augmented Generation (RAG) đã ra đời. Bằng cách cung cấp trực tiếp những kiến thức thực tế từ cơ sở dữ liệu bên ngoài, hệ thống sử dụng RAG đảm bảo mô hình tư duy dựa trên sự thật chứ không hoàn toàn dựa vào trọng số nội tại.

Mặc dù xuất hiện như một điểm sáng, mô hình RAG truyền thống (Naive RAG) mới chỉ giải quyết tốt bài toán truy vấn đơn lượt (single-turn QA). Khác với hỏi - đáp đơn lượt nơi mỗi truy vấn mang đầy đủ ngữ cảnh độc lập, hội thoại đa lượt (multi-turn conversation) là một chuỗi tương tác liên tục mang tính kế thừa. Khi ứng dụng vào môi trường này, RAG bộc lộ rõ điểm yếu do bản chất phi trạng thái (stateless) của nó: xử lý mỗi câu hỏi như một thực thể cô lập. Cụ thể, hệ thống phải đối mặt với ba rào cản lớn:

- Sự mơ hồ về ngữ nghĩa và Tỉnh lược từ (Coreference and Omission):** Người dùng có xu hướng giao tiếp tự nhiên bằng cách sử dụng các đại từ thay thế (nó, kia, anh ấy, phương pháp đó...) hoặc lược bỏ hoàn toàn chủ ngữ vì cho rằng hệ thống đã "hiểu" ngữ cảnh.
- Chuyển đổi chủ đề ngầm (Implicit Topic Shifts):** Trong một phiên tương tác dài, người dùng có thể rẽ nhánh chủ đề hoặc đột ngột quay lại một vấn đề đã thảo luận từ nhiều lượt trước.
- Quá tải cửa sổ ngữ cảnh (Context Window Overflow):** Phương pháp ngây thơ nhất để giữ mạch hội thoại là ghép nối toàn bộ lịch sử đoạn chat vào câu hỏi hiện tại. Tuy nhiên, điều này làm lãng phí lượng lớn token, đẩy hệ thống chạm ngưỡng giới hạn đầu vào của LLM.

Nhận thức rõ khoảng trống này, nhóm nghiên cứu nhận định rằng cốt lõi để nâng cấp hệ thống Conversational AI không nằm ở việc sử dụng một LLM sinh văn bản mạnh hơn, mà phải nằm ở việc tối ưu hóa module Retrieval. Báo cáo này tập trung vào các mục tiêu then chốt: (1) Phân tích cơ chế gây lỗi của bộ Retriever trong môi trường hội thoại đa lượt; (2) Đề xuất và cài đặt các giải pháp tối ưu xử lý ngữ cảnh

tiền truy xuất (Pre-retrieval optimization); và (3) Đánh giá hiệu năng của các phương pháp nhằm hoàn thiện một pipeline Conversational RAG có khả năng duy trì độ chính xác cao và trải nghiệm liền mạch cho người dùng cuối.

II. MÔ TẢ BÀI TOÁN

Để phân tích bản chất của các giới hạn hiện tại và làm tiền đề xây dựng các phương pháp tối ưu, bài toán RAG trong hội thoại đa lượt được đặc tả hình thức hóa với các thành phần cốt lõi như sau:

A. Các biến đầu vào

- Kho tri thức (Knowledge Base):** Tập hợp các tài liệu tri thức:

$$\mathcal{D} = \{d_1, d_2, \dots, d_N\}$$

- Lịch sử hội thoại (Dialogue History):** Chuỗi các lượt tương tác tính đến thời điểm t , ký hiệu là:

$$H_t = \{(u_1, r_1), (u_2, r_2), \dots, (u_{t-1}, r_{t-1})\}$$

với u_i là truy vấn của người dùng và r_i là phản hồi tương ứng của hệ thống.

- Truy vấn hiện tại (Current Query):** Câu hỏi ở lượt thứ t , ký hiệu là: u_t

B. Mô hình hóa quá trình xử lý

Kiến trúc RAG đa lượt hoạt động dựa trên ba giai đoạn toán học tuần tự:

- Đầu tiên, hệ thống thực hiện hàm biến đổi ngữ cảnh f_θ để tổng hợp thông tin từ câu hỏi u_t và lịch sử H_t , tạo ra một biểu diễn truy vấn tối ưu q_t^* :

$$q_t^* = f_\theta(u_t, H_t) \quad (1)$$

- Sau đó, hàm truy xuất g_ϕ sử dụng q_t^* để trích xuất tập con K tài liệu phù hợp nhất (D_t^*) từ kho tri thức $\mathcal{D}$:

$$D_t^* = g_\phi(q_t^*, \mathcal{D}) \quad (2)$$

- Cuối cùng, mô hình sinh văn bản p_ψ tính toán để đưa ra phản hồi mục tiêu r_t :

$$r_t = \arg \max_r p_\psi(r | q_t^*, H_t, D_t^*) \quad (3)$$

C. Mục tiêu tối ưu

Nhằm đạt được mục tiêu cuối cùng là tối ưu hóa chất lượng của phản hồi đầu ra r_t , nhiệm vụ trọng tâm của các kiến trúc RAG bao gồm 3 khía cạnh:

- 1) Nâng cao độ chính xác của biểu diễn truy vấn q_t^* .
- 2) Lưu trữ và khai thác hiệu quả lịch sử hội thoại H_t trong phiên làm việc kéo dài.
- 3) Truy xuất tập tài liệu D_t^* nhanh, đúng và đủ.

III. KHẢO SÁT CÁC PHƯƠNG PHÁP HIỆN CÓ

Kiến trúc RAG bao gồm hai khía cạnh cốt lõi: (1) **Tối ưu hóa Kho tri thức và Truy xuất tài liệu** và (2) **Quản lý Ngữ cảnh và Bộ nhớ hội thoại**. Trong khi khía cạnh thứ nhất đã được khai thác triệt để ở các hệ thống hỏi - đáp đơn lượt (single-turn) thông qua các thuật toán tìm kiếm và re-ranking, rào cản lớn nhất đối với môi trường đa lượt lại nằm ở khía cạnh thứ hai: khả năng duy trì mạch hội thoại ổn định.

Dựa trên đặc tả hình thức đã đề cập, các nghiên cứu về đa lượt hiện nay tập trung giải quyết ba mục tiêu tối ưu chính: làm rõ biểu diễn truy vấn q_t^* , nâng cao độ chính xác của tập tài liệu truy xuất D_t^* , và kiểm soát hiệu quả không gian lưu trữ lịch sử hội thoại H_t . Từ ba mục tiêu này, các phương pháp hiện hành được hệ thống hóa thành ba hướng tiếp cận cốt lõi sau đây:

- **Tái cấu trúc truy vấn (Query Rewriting / Reformulation):**

Hướng tiếp cận này trực tiếp giải quyết mục tiêu tối ưu hóa q_t^* thông qua khâu tiền xử lý. Hệ thống sử dụng một mô hình ngôn ngữ phụ trợ để khôi phục các đại từ bị tỉnh lược và diễn dịch lại câu hỏi của người dùng dựa trên lịch sử hội thoại.

→ *Khung kiến trúc: RRR (Rewrite-Retrieve-Read).*

- **Quản lý và Tinh lọc bộ nhớ (Memory Management & Filtering):**

Nhằm giải quyết bài toán kiểm soát H_t , hướng tiếp cận này can thiệp trực tiếp vào cách tổ chức và lưu trữ lịch sử tương tác. Thay vì đẩy toàn bộ chuỗi chat thô vào cửa sổ ngữ cảnh, hệ thống chủ động đánh giá để chỉ giữ lại các "điểm neo" quan trọng, phân mảnh sự kiện hoặc loại bỏ thông tin gây nhiễu. Điều này giúp duy trì bộ nhớ dài hạn mà không làm tràn tài nguyên tính toán của mô hình.

→ *Tiêu biểu: MemoChat, HingeMem và EMem*

- **Cấu trúc hóa tri thức hội thoại (Knowledge Structuring):**

Để tối đa hóa chất lượng của tập tài liệu D_t^* , hướng đi này tạo ra sự giao thoa trực tiếp giữa bộ nhớ hội thoại và các thuật toán đồ thị. Thay vì lưu trữ H_t dưới dạng văn bản tuyến tính, phương pháp này chuyển đổi lịch sử tương tác thành các mạng lưới logic (diễn hình là Đồ thị tri thức). Cơ chế này mô phỏng cách não bộ liên kết thông tin, cho phép thực hiện suy luận đa bước (multi-hop reasoning) dọc theo các thực thể.

→ *Các cách tiếp cận: StructMem, HippoRAG và EMem-G*

Các tiểu mục tiếp theo sẽ đi sâu phân tích kiến trúc kỹ thuật, ưu điểm và điểm nghẽn của từng phương pháp nổi bật, tạo tiền đề khách quan cho việc đề xuất một kiến trúc kết hợp (hybrid) hiệu quả hơn trong phần sau của báo cáo.

A. RRR

Theo quy trình thông thường, các hệ thống RAG hoạt động dựa trên cơ chế "Truy xuất rồi Đọc" (Retrieve-then-Read). Trong cơ chế này, hệ thống vận hành theo một đường ống tuyến tính: nhận trực tiếp truy vấn thô của người dùng, biến đổi thành vector nhúng để tìm kiếm tài liệu tương đồng trong kho tri thức, sau đó ghép nối tài liệu tìm được vào prompt để LLM sinh câu trả lời. Cơ chế này hoạt động hiệu quả đối với các tác vụ đơn lượt nhưng không còn phù hợp để đối mặt với tính liên tục và phụ thuộc ngữ cảnh của hội thoại đa lượt. Nhằm khắc phục điểm yếu này, Ma và cộng sự [1] đã đề xuất khung kiến trúc Rewrite-Retrieve-Read (RRR).

1) Ý tưởng cốt lõi:

Triết lý của phương pháp này xuất phát từ việc giải quyết tận gốc nguyên nhân gây nhiễu: chất lượng của truy vấn đầu vào. Thay vì cố gắng tìm kiếm bằng một câu hỏi mơ hồ hoặc chứa các đại từ thay thế, hệ thống sẽ khôi phục ngữ cảnh từ lịch sử giao tiếp để "viết lại" truy vấn thô thành một phiên bản tường minh và độc lập. Việc chuẩn hóa đầu vào này mang lại tác động kép: giúp thuật toán rà soát kho tri thức chính xác hơn, đồng thời cung cấp bối cảnh rõ ràng để mô hình ngôn ngữ sinh phản hồi chất lượng cao ở bước cuối cùng.

2) Phương pháp triển khai:

Kiến trúc RRR vận hành thông qua ba thành phần nối tiếp, tương ứng với các giai đoạn đã được đặc tả trong quy trình xử lý tổng quát:

- **Bộ Viết lại (Query Rewriter):** Đóng vai trò thực thi hàm tiền xử lý f_θ theo mô tả của công thức (1). Nó tiếp nhận truy vấn thô u_t cùng lịch sử hội thoại H_t để diễn dịch và trả về biểu diễn truy vấn đã được tái cấu trúc q_t^* .
- **Bộ Truy xuất (Retriever):** Tương ứng với hàm tìm kiếm g_ϕ tại công thức (2). Thành phần này sử dụng truy vấn q_t^* làm "mỏ neo" để trích xuất tập tài liệu liên quan nhất D_t^* từ kho tri thức $\mathcal{D}$.
- **Bộ Đọc (Reader):** Là mô hình sinh văn bản cuối cùng cùng p_ψ như công thức (3). Nó tổng hợp thông tin từ câu hỏi, lịch sử và nguồn tri thức để đưa ra phản hồi cuối cùng r_t .

Nhìn chung, khung kiến trúc RRR đóng vai trò là một nền tảng thiết yếu, thiết lập cấu trúc cơ sở cho việc tối ưu hóa quy trình truy xuất trong hội thoại đa lượt. Đây là tiền đề để áp dụng và tích hợp linh hoạt các phương pháp xử lý ngữ cảnh chuyên sâu sẽ được trình bày chi tiết ở những tiểu mục tiếp theo.

B. MemoChat

Khung kiến trúc RRR không đưa ra một cơ chế cụ thể nào để đối phó với sự phình to của lịch sử truy vấn H_t .

Để giải quyết khoảng trống này, Tan và cộng sự [2] đã đề xuất MemoChat.

1) Ý tưởng và hướng tiếp cận:

Thay vì bảo lưu toàn bộ văn bản thô như hệ thống RAG cơ bản, MemoChat lấy cảm hứng từ thói quen "ghi chép" (take notes) của con người. Hướng tiếp cận cốt lõi là tinh chỉnh (fine-tuning) các mô hình ngôn ngữ để chúng tự động chất lọc, tóm tắt các cụm hội thoại cũ thành các "bản ghi chú" (memos) có cấu trúc. Các bản ghi chú này sẽ đóng vai trò là bộ nhớ dài hạn, đại diện cho lịch sử H_t một cách cô đọng nhất.

2) Phương pháp triển khai:

Theo kiến trúc đề xuất trong [2], hệ thống MemoChat duy trì mạch hội thoại thông qua một quy trình (pipeline) lặp lại gồm ba giai đoạn:

- **Ghi nhớ (Memorization):** Khi bộ nhớ ngắn hạn (các lượt tương tác gần nhất) đạt đến giới hạn token quy định, hệ thống kích hoạt một tác vụ nền để đọc lại đoạn hội thoại này, trích xuất các thông tin về chủ đề, sự kiện, sở thích người dùng và nén chúng thành một bản ghi chú mới để lưu trữ vào cơ sở dữ liệu.
- **Truy xuất (Retrieval):** Khi nhận được truy vấn u_t , hệ thống sử dụng chính câu hỏi này (hoặc truy vấn đã được tái cấu trúc) để tìm kiếm các bản ghi chú liên quan nhất trong quá khứ.
- **Phản hồi (Response Generation):** Mô hình sinh văn bản sẽ nhận một prompt tổng hợp bao gồm: Tập hợp các bản ghi chú được truy xuất, ngữ cảnh hội thoại ngắn hạn và truy vấn u_t , từ đó đưa ra câu trả lời cuối cùng.

Để tối ưu hóa việc quản lý và đối chiếu, thay vì lưu trữ văn bản tự do, mỗi bản ghi chú trong hệ thống được tổ chức theo một cấu trúc dữ liệu rõ ràng. Cấu trúc này được minh họa chi tiết trong Bảng I.

Bảng I
CẤU TRÚC DỮ LIỆU CỐT LÕI CỦA MỘT BẢN GHI CHÚ (MEMO)

Trường dữ liệu	Mô tả chức năng	Ví dụ
Chủ đề	Từ khóa chính để đối chiếu vector truy xuất.	quantum physics, businesss,...
Tóm tắt	Lưu trữ cốt lõi thông tin dưới dạng văn bản ngắn gọn, loại bỏ ngôn ngữ giao tiếp dư thừa.	Bot explains Super-position ...
Đoạn hội thoại	Ghi lại nội dung từng lượt trao đổi cụ thể.	User: asked... Bot: replied...

3) Ưu nhược điểm:

Ưu điểm:

- **Khắc phục quá tải bộ nhớ (Context Window Overflow):** Bằng cách tóm tắt nội dung theo chủ đề, Memochat giúp mô hình không "quên" các chi tiết quan trọng.
- **Bảo toàn tính nhất quán (Long-range Consistency):** Các bản ghi chú hoạt động như

một bộ khung kiến thức nền tảng về người dùng. Điều này giúp hệ thống phản hồi ổn định, giữ vững tính cách và mạch câu chuyện xuyên suốt cuộc hội thoại dài.

- **Giảm độ trễ phản hồi (Latency):** Hạ thấp chi phí tính toán ở bước cuối cùng nhờ giảm số lượng token cần tái xử lý, qua đó tăng tốc độ sinh văn bản của mô hình.

Nhược điểm:

- **Sự mất mát thông tin chi tiết (Lossy Compression):** Quá trình LLM tự động tóm tắt văn bản mang bản chất của một phép nén có tổn hao. Trong quá trình "ghi chú", hệ thống dễ dàng lược bỏ các tiểu tiết cụ thể (fine-grained details) hoặc làm phai nhạt các sắc thái cảm xúc, dụng ý ngầm trong lời nói nguyên bản của người dùng.
- **Gia tăng chi phí xử lý nền (Computational Overhead):** Việc kích hoạt quá trình tạo Memos theo một chu kỳ tính yêu cầu một lượng tài nguyên tính toán bổ sung để duy trì cấu trúc.
- **Nhiều chủ đề cục bộ (Topic Entanglement):** Do việc đóng gói memo bị phụ thuộc vào chu kỳ (giới hạn token/số lượt hội thoại) thay vì sự thay đổi ngữ nghĩa sự kiện, một bản ghi chú có thể vô tình bị ép gộp chung nhiều chủ đề đang thảo luận đang dở cùng lúc. Điều này làm giảm độ hiệu quả của từ khóa và bản tóm tắt được tạo ra.
- **Hạn chế về khả năng mở rộng (Scalability Bottleneck):** Theo thiết kế, hệ thống liên tục tạo thêm Memo mới và đẩy vào cơ sở dữ liệu để truy xuất Top- K . Khi số lượng Memo tăng lên trong các phiên hội thoại siêu dài, MemoChat thiếu vắng một cơ chế bảo trì tự động (như tóm tắt phân cấp - hierarchical summarization hoặc gộp các Memo cũ), dẫn đến nguy cơ phân mảnh và làm loãng độ chính xác của tập tài liệu truy xuất.

C. HingeMem

Mặc dù MemoChat thiết lập được một bộ nhớ dài hạn dựa trên các bản ghi chú, tuy nhiên việc tóm tắt theo chu kỳ tính vẫn gây ra sự dư thừa tài nguyên tính toán và rủi ro nhiều chủ đề cục bộ. Để tối ưu hóa quá trình này, Zhong và Gao [3] đã đề xuất kiến trúc HingeMem nhằm mục đích cá nhân hóa việc ghi nhớ và truy xuất theo biến động thực tế của hội thoại.

1) Ý tưởng và hướng tiếp cận:

HingeMem thay đổi triết lý lưu trữ từ định lượng (dựa trên số lượng token) sang định tính, bằng cách kế thừa Lý thuyết phân đoạn sự kiện (Event Segmentation Theory) trong khoa học nhận thức. Ý tưởng cốt lõi là hệ thống không ghi chép thông tin liên tục mà chỉ tạo ra các "điểm neo" (hinges) khi phát hiện ra sự thay đổi rõ rệt về bối cảnh hội thoại. Hướng tiếp cận này giúp loại bỏ sự dư thừa trong việc cập nhật bộ nhớ thường xuyên, đồng thời chuyển đổi cơ chế truy xuất từ Top- K tính sang trạng thái thích ứng (Adaptive) hoàn toàn với truy vấn hiện tại.

2) Phương pháp triển khai:

Hệ thống HingeMem vận hành mạch hội thoại thông qua hai cơ chế chính nhằm tối ưu hóa đồng thời cả quá trình ghi vào và đọc ra từ bộ nhớ:

- **Ghi nhớ kích hoạt theo biên (Boundary-triggered Writing):** Hệ thống liên tục theo dõi bối cảnh thông qua một không gian trạng thái sự kiện, ký hiệu là $\mathcal{E}_t$. Trạng thái này được cấu thành từ 4 thành phần (elements) định danh ngữ cảnh:

$$\mathcal{E}_t = \{e_{person}, e_{time}, e_{location}, e_{topic}\}$$

Trong đó:

- e_{person} : đại diện cho các thực thể
- e_{time} : là mốc thời gian
- $e_{location}$: là không gian sự kiện
- e_{topic} : là chủ đề thảo luận.

Quá trình đóng gói toàn bộ chuỗi hội thoại vừa diễn ra thành một phân đoạn (segment) duy nhất chỉ kích hoạt khi hệ thống phát hiện "biên sự kiện" bị phá vỡ, hay sự dịch chuyển trạng thái $\Delta\mathcal{E} \neq 0$ giữa các lượt tương tác.

- **Truy xuất thích ứng truy vấn (Query-Adaptive Retrieval):** Thay vì sử dụng Top- K cố định, HingeMem triển khai quy trình truy xuất động:

- * **Định tuyến thích ứng (Adaptive Routing):** Phân tích *Truy vấn hiện tại* tại u_t để điều hướng trực tiếp đến các phần tử $e \in \mathcal{E}$ liên quan, thu hẹp phạm vi tìm kiếm.
- * **Ngừng truy vấn thích ứng (Adaptive Stop):** Hệ thống tự động đánh giá độ phức tạp của câu hỏi để quyết định độ sâu truy xuất *Kho tri thức* và chủ động dừng khi tập tài liệu D_t^* đã đủ thông tin, giúp giảm nhiễu và thời gian truy xuất.

3) Ưu nhược điểm:

Ưu điểm:

- **Tối ưu hóa triệt để chi phí vận hành:** Loại bỏ các thao tác tóm tắt lặp lại không cần thiết nhờ cơ chế kích hoạt theo biên sự kiện.
- **Bảo toàn tính nhất quán và toàn vẹn chủ đề:** Việc lưu trữ theo phân đoạn sự kiện giúp mỗi bản ghi nhớ chứa đựng trọn vẹn một ngữ cảnh logic, không bị cắt xén cơ học.
- **Truy xuất tinh gọn:** Cơ chế Adaptive Retrieval đảm bảo tập tài liệu D_t^* luôn đạt trạng thái "vừa đủ", tránh hiện tượng loãng thông tin.

Nhược điểm:

- **Rủi ro phân mảnh do nhận diện sai biên:** Nếu mô hình phân loại sai sự chuyển dịch của tập $\mathcal{E}$, mạch logic hội thoại sẽ bị chia cắt không hợp lý, gây khó khăn cho việc truy xuất.
- **Phụ thuộc vào năng lực suy luận của mô hình nền tảng:** Các tác vụ định tuyến thích ứng và quyết định điểm dừng đòi hỏi LLM phải có khả năng lập luận (reasoning) cực kỳ sắc bén. Nếu sử dụng các

mô hình ngôn ngữ nhỏ hoặc yếu, các cơ chế thích ứng này rất dễ hoạt động sai lệch, làm sụt đổ toàn bộ hiệu năng của hệ thống.

IV. ĐỀ XUẤT HƯỚNG TIẾP CẬN

Trong nghiên cứu này, chúng em đề xuất hai hướng tiếp cận nhằm tái cấu trúc truy vấn hội thoại đa lượt thành truy vấn độc lập Q_{final} , bao gồm: (i) **State-Centric Adaptive Pipeline** và (ii) **Compression-Centric Pipeline**. Hai phương pháp khác nhau ở cách quản lý và khai thác ngữ cảnh hội thoại, trong đó hướng thứ nhất ưu tiên kiểm soát trạng thái, còn hướng thứ hai tập trung vào nén và truy xuất lịch sử.

Trong mỗi lượt truy vấn, quy trình xử lý sẽ bắt đầu với một nhóm đầu vào chung cơ bản bao gồm:

- u_t : Truy vấn hiện tại của người dùng
- H_t^{short} : Bộ nhớ ngắn hạn (Active Chat) chứa các lượt tương tác gần nhất, chưa được nén, giới hạn trong ngưỡng token cho phép $[0, \tau_{token}]$.
- $\mathcal{M}$: Kho trí nhớ Memos Database (tập con của kho tri thức $\mathcal{D}$), lưu trữ các bản ghi chú được nén dựa trên cơ chế của MemoChat [2]

Và đầu ra mong muốn:

- q_t^* : Truy vấn được viết lại, đầy đủ thông tin định danh, có tính độc lập cao, sẵn sàng cho mô hình Reader cuối cùng xử lý và đưa ra câu trả lời chính xác.

A. Hướng tiếp cận 1: State-Centric Adaptive Pipeline

Hướng tiếp cận này thiết lập một cơ chế quản lý ngữ cảnh dựa trên trạng thái sự kiện (state-based), trong đó việc truy xuất bộ nhớ dài hạn được điều khiển linh hoạt bởi nhu cầu thực tế của truy vấn hiện tại. Mục tiêu cốt lõi là duy trì một không gian trạng thái tinh gọn, có cấu trúc thay vì lưu trữ lịch sử hội thoại thô để tránh hiện tượng quá tải cửa sổ ngữ cảnh.

Trong đó, một không gian trạng thái $\mathcal{S}$ được cấu tạo bao 4 thành phần cốt lõi:

- Các thực thể (entities).
- Thuộc tính (attributes).
- Các ràng buộc người dùng (constraints).
- Ý định hội thoại (intent).

Như vậy, ta sẽ có thêm một đầu vào bổ sung đại diện cho ngữ cảnh sự kiện từ lượt hội thoại trước đó.

$$\mathcal{S}_{t-1}: \text{Trạng thái ngữ cảnh tại lượt } t-1 \text{ trước đó}$$

Quy trình tái cấu trúc truy vấn được vận hành qua 5 giai đoạn chính:

1) Phát hiện biên sự kiện (Boundary Detection):

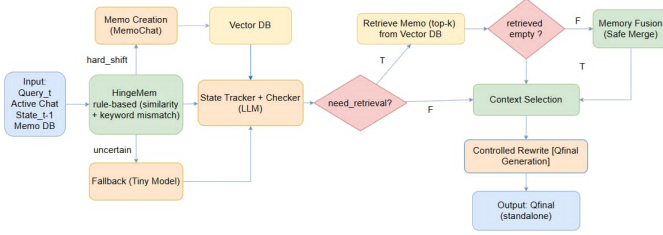
Dựa trên cơ chế Boundary-guided Memory đề xuất trong HingeMem [3], hệ thống định vị ranh giới thay đổi bối cảnh theo ba kịch bản:

- **Hard shift:** Xảy ra khi tín hiệu rule-based (độ tương đồng ngữ nghĩa, sự sai lệch tập thực thể) chắc chắn hoặc đoạn chat đạt giới hạn token. Lúc này, hệ thống thực hiện đóng gói H_t^{short} vào kho lưu trữ $\mathcal{M}$:

$$m_k = \text{Summarize}(H_t^{short})$$

$$\mathcal{M} \leftarrow \mathcal{M} \cup \{m_k\}$$

- **Soft shift:** Khi các tín hiệu ranh giới nằm trong vùng không chắc chắn, một mô hình dự phòng (Fallback Model) sẽ thay thế cơ chế rule-based đưa ra quyết định cuối cùng. Mô hình này được hạn chế sử dụng để tối ưu chi phí tính toán và độ trễ.
- **Continue:** tiếp tục tiến trình xử lý, bỏ qua thực hiện lưu trữ bổ sung.



Hình 1. Sơ đồ luồng xử lý State-Centric với cơ chế kiểm soát trạng thái.

2) Cập nhật và Kiểm tra Trạng thái (State Tracking & Checker):

Kiểm tra đồng tham chiếu (coreference resolution) và duy trì tính nhất quán theo nguyên tắc ưu tiên thông tin mới (recency priority). Hàm Φ nội suy ra trạng thái mới và cờ điều kiện truy xuất δ_{ret} :

$$(\mathcal{S}_t, \delta_{ret}) = \Phi(\mathcal{S}_{t-1}, H_t^{short}, u_t)$$

Để tối ưu hóa hiệu năng, quá trình cập nhật và kiểm tra điều kiện truy xuất được thực hiện song song thông qua kỹ thuật multi-task prompting trong một lượt suy luận duy nhất, giúp đảm bảo tốc độ phản hồi cho hệ thống.

3) Truy xuất bộ nhớ (Memory Retrieval):

Khi cờ $\delta_{ret} = 1$, hệ thống thực hiện truy xuất Top-K các bản ghi chú liên quan nhất M_t^* từ kho $\mathcal{M}$ thông qua thuật toán tìm kiếm vector

$$\mathcal{M}_t^* = g_\phi(\mathcal{S}_t, \mathcal{M})$$

4) Dung hợp và Tinh lọc ngữ cảnh (Memory Fusion & Context Selection):

Các thông tin truy xuất được $\mathcal{M}_t^*$ tích hợp vào trạng thái $\mathcal{S}_t$ hiện tại theo nguyên tắc an toàn, chỉ bổ sung các slot còn thiếu và không ghi đè thông tin đã tồn tại, nhằm đảm bảo tính nhất quán của trạng thái.

$$\mathcal{S}_t^{fused} = \mathcal{S}_t \oplus \mathcal{M}_t^*$$

Sau đó, bộ lọc Selection sẽ ưu tiên các thông tin có độ tin cậy cao để tạo ra biểu diễn tối ưu $\mathcal{S}_t^*$:

$$\mathcal{S}_t^* = \text{Selection}(\mathcal{S}_t^{fused})$$

5) Tái cấu trúc truy vấn có kiểm soát (Controlled Query Rewrite):

Sử dụng mô hình ngôn ngữ thực hiện biến đổi truy vấn thô u_t dựa trên không gian trạng thái tinh lọc $\mathcal{S}_t^*$:

$$q_t^* = f_\theta(u_t, \mathcal{S}_t^*)$$

→ Kết quả cuối cùng của quy trình là một biểu diễn truy vấn q_t^* đầy đủ thông tin như đã đề ra.

B. Hướng tiếp cận 2: Compression-Centric Pipeline

Khác với hướng tiếp cận dựa trên trạng thái tường minh, Compression-Centric Pipeline tập trung vào việc quản lý tài nguyên thông qua cơ chế nén bối cảnh chủ động. Phương pháp này tối ưu hóa cửa sổ ngữ cảnh bằng cách chuyển đổi lịch sử hội thoại thô thành các bản ghi nhớ (Memos) dựa trên kiến trúc MemoChat [2] và thực hiện truy xuất dựa trên các thực thể điểm nhấn nhằm tái cấu trúc truy vấn q_t^* .

Trong quy trình này, hệ thống sử dụng thêm các biến định danh đặc thù sau:

$\mathcal{E}_t$: Tập hợp các thực thể (Entities) được trích xuất làm "mỏ neo" ngữ nghĩa.

Quy trình vận hành của hệ thống được thực hiện qua các giai đoạn tuần tự sau:

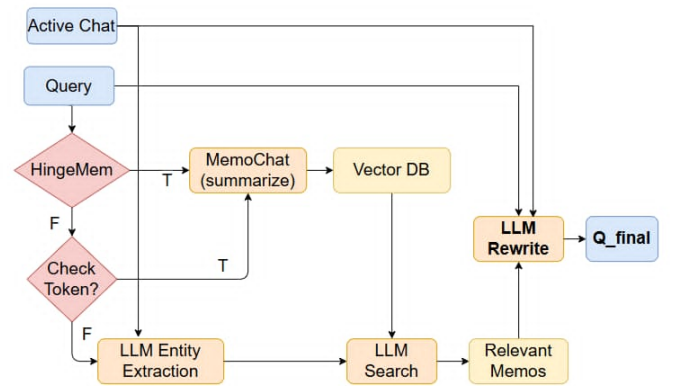
1) Logic điều phối kép (Dual Gating Logic):

Hệ thống đánh giá đồng thời ranh giới sự kiện (thông qua cơ chế HingeMem [3]) và ngưỡng giới hạn token τ_{token} để quyết định có nên tiến hành nén ngữ cảnh hiện tại hay không.

Nếu quá trình lưu trữ (**Memo Creation**) được khởi động, toàn bộ nội dung trong H_t^{short} được mô hình ngôn ngữ chất lọc thành bản ghi chú m_k có cấu trúc, sau đó được cập nhật vào kho lưu trữ dài hạn:

$$m_k = \text{Summarize}(H_t^{short})$$

$$\mathcal{M} \leftarrow \mathcal{M} \cup \{m_k\}$$



Hình 2. Sơ đồ luồng xử lý Compression-Centric tập trung vào nén bối cảnh.

2) Trích xuất thực thể (Entity Extraction):

Nhằm tối ưu hóa độ chính xác cho khâu tìm kiếm, hệ thống trích xuất tập thực thể $\mathcal{E}_t$ từ truy vấn hiện tại và bối cảnh gần để làm mờ neo định vị thông tin:

$$\mathcal{E}_t = \text{Extract}(u_t, H_t^{\text{short}})$$

3) Truy xuất dựa trên thực thể (Memory Retrieval):

Sử dụng tập $\mathcal{E}_t$ để truy vấn các bản ghi nhớ liên quan nhất M_t^* từ kho $\mathcal{M}$ (tương ứng với hàm truy xuất g_ϕ tại công thức (2)):

$$M_t^* = g_\phi(\mathcal{E}_t, \mathcal{M})$$

4) Tái cấu trúc truy vấn (Contextual Reformulation):

Khởi thực thi hàm f_θ (định nghĩa tại công thức (1)) thực hiện tổng hợp thông tin từ truy vấn gốc, bộ nhớ ngắn hạn và các bản ghi nhớ liên quan để tạo ra truy vấn độc lập cuối cùng:

$$q_t^* = f_\theta(u_t, H_t^{\text{short}}, M_t^*)$$

→ Kết quả đầu ra q_t^* được tối ưu hóa dựa trên việc nhúng các thực thể lịch sử quan trọng, đảm bảo tính đầy đủ về mặt thông tin mà không cần duy trì các cấu trúc trạng thái phức tạp.

C. So sánh hai hướng tiếp cận

Hai phương án đề xuất thể hiện hai chiến lược khác nhau trong việc xử lý ngữ cảnh hội thoại:

- **State-Centric Pipeline** cho phép kiểm soát ngữ cảnh tốt hơn nhờ biểu diễn trạng thái tường minh, đồng thời giảm phụ thuộc vào truy xuất lịch sử và hạn chế nhiễu trong hội thoại dài.
- **Compression-Centric Pipeline** có cấu trúc đơn giản hơn, dễ triển khai, nhưng phụ thuộc nhiều hơn vào chất lượng truy xuất và khả năng tổng hợp của mô hình ngôn ngữ.

Hướng tiếp cận thứ hai có thể được xem như một baseline tốt để đánh giá hiệu quả của kiến trúc State-Centric trong các kịch bản hội thoại phức tạp.

V. PHƯƠNG PHÁP ĐÁNH GIÁ

A. Tập dữ liệu (Dataset)

Để đánh giá khả năng truy xuất ngữ cảnh trong nhiều turn nhóm đề xuất sử dụng bộ dữ liệu đánh giá LOCOMO [4]. Tập dữ liệu này có định json dễ tiếp cận. Bộ dữ liệu bao gồm 50 cuộc hội thoại độc lập nhau giữa hai người với trung bình 3000 lượt nói và 9000 từ. Các thành phần chính:

Sample_id	: Mã định danh của mẫu
conversation	: Lưu trữ cuộc trò chuyện của 2 người, có dữ liệu thời gian tham chiếu
observation	: Các bản tóm tắt cuộc trò chuyện
session_summary	: 1 đoạn ngắn tóm tắt cuộc trò chuyện
qa	: Danh sách các câu hỏi

Điểm nổi bật của tập dữ liệu này nằm ở việc nó được mô phỏng như một cuộc trò chuyện thực tế. Phân chia bộ câu hỏi thành 5 thành phần:

- 1) **Single question**: Tập các câu hỏi chỉ cần truy xuất dữ liệu từ một lượt trả lời là có đủ thông tin.
- 2) **Mutiple question**: Tập các câu hỏi yêu cầu phải truy vấn nhiều hơn 1 phiên thì mới đủ thông tin để trả lời.
- 3) **Temporal**: Tập các câu hỏi yêu cầu RAG không chỉ truy xuất đúng mà còn cần suy luận dựa trên dữ liệu truy xuất mới có thể trả lời (Ví dụ: Nội dung truy xuất được "Anna đi thăm bạn ngày hôm qua", mô hình phải dựa vào thời gian hội thoại để đưa ra ngày chính xác.)
- 4) **Commonsense or world knowledge**: Các câu hỏi phải kết hợp các kiến thức hiện tại mới có thể trả lời được (Ví dụ: ...)
- 5) **Adversarial**: Các câu hỏi đối kháng để đánh lừa mô hình đưa ra câu trả lời sai.

Đây là bộ benchmark đánh giá với đầy đủ input và expected output cho việc đánh giá mô hình ứng dụng RAG trong bài toán hội thoại nhiều lượt.

B. LOCOMO

Dựa trên cơ sở bộ dữ liệu đã phát triển, LOCOMO đề xuất một phương thức đánh giá kết hợp dựa trên các chỉ số: **BLEU score**, **F1 score** và **LLM score**.

Quy trình đánh giá thực nghiệm

Quy trình đánh giá được thực hiện qua ba bước chính nhằm đảm bảo tính khách quan và chính xác cho các mô hình RAG:

Bước 1: Giả lập ngữ cảnh hội thoại. Đầu tiên, hệ thống sẽ tái dựng lại kịch bản cuộc trò chuyện từ bộ dữ liệu. Các lượt thoại được sắp xếp theo đúng trình tự thời gian dựa trên thông tin thời gian thực (*data_time*), tạo ra một kho dữ liệu nền tảng để mô hình bắt đầu truy xuất.

Bước 2: Áp dụng RAG để xử lý câu hỏi. Tại bước này, mô hình sẽ thực hiện việc tìm kiếm thông tin tùy theo độ phức tạp của câu hỏi.

Bước 3: Tổng hợp kết quả và đánh giá. Cuối cùng, hệ thống sẽ thu thập câu trả lời thực tế từ mô hình để đối chiếu với đáp án gợi ý có sẵn trong bộ benchmark. Dựa trên sự tương đồng giữa đầu ra của máy và đáp án chuẩn, các chỉ số như BLEU, F1 và LLM score sẽ được tính toán để đưa ra kết luận cuối cùng về hiệu năng của mô hình.

C. RAGAS

Kết quả đánh giá của LoCoMo trên nhiều mô hình khác nhau cho kết quả tương đối kém, chủ yếu đạt 20–40%. Kết quả thực nghiệm cho thấy các LLM gặp nhiều thách thức trong việc hiểu những cuộc hội thoại dài và nắm bắt các mối quan hệ nhân quả, thời gian trong đó. Điều này cho thấy khả năng phân hóa giữa các mô hình không cao, nếu đa phần đều cho kết quả thấp đồng đều ở mức 20–40%, thì bộ benchmark thiếu đi sức mạnh phân biệt để đánh giá tiến bộ thực sự.

RAGAS [5] tiếp cận bài toán đánh giá khả năng nhớ ngữ cảnh theo hướng câu chuyện có cấu trúc (structured narrative): thay vì chỉ dùng hội thoại thô như LoCoMo, RAGAS tổ chức thông tin đầu vào thành các đơn vị ngữ nghĩa rõ ràng — tương

tự như các "chương" trong một câu chuyện — để kiểm tra xem hệ thống RAG có thực sự nhớ đúng, nhớ đủ và nhớ có ưu tiên hay không.

Cũng tương tự như LOCOMO, lí do chúng tôi lựa chọn RAGAS trở thành bộ đánh giá thứ hai vì hai lí do chính. Thứ nhất, RAGAS cung cấp bộ mã nguồn mở giúp thiết lập kiến trúc sử dụng RAG một cách chi tiết, đồng thời hướng dẫn sinh bộ benchmark cho bộ dữ liệu đó. Thứ hai, RAGAS được công nhận, sử dụng và đóng góp với đông đảo người dùng.

RAGAS cung cấp nhiều chỉ số đánh giá gồm context recall, context precision, faithfulness, Faithfulness,... . Trong đó, hai phương pháp đánh giá nhóm dự định sử dụng chính bao gồm:

- **context recall:** Chỉ số này tập trung vào việc không bỏ sót các kết quả quan trọng — recall cao đồng nghĩa với ít tài liệu liên quan bị bỏ lỡ hơn.

$$\text{Context Recall} = \frac{\text{Số lượng ngữ cảnh được truy xuất}}{\text{Tổng số ngữ cảnh tham chiếu}} \quad (4)$$

- **context precision:** Chỉ số này đánh giá mức độ mà các đoạn liên quan trong ngữ cảnh được truy xuất được đặt ở vị trí đầu trong bảng xếp hạng

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\text{Tổng số mục trong top } K} \quad (5)$$

Về mặt định lượng, các kết quả thực nghiệm điển hình trên pipeline RAG với RAGAS cho thấy: faithfulness đạt 0.955, answer relevancy 0.923, context recall 0.917, nhưng context precision chỉ đạt 0.733 và answer correctness 0.666 — cho thấy rõ ràng rằng ngay cả khi hệ thống nhớ đủ thông tin (recall cao), vị trí và mức độ ưu tiên của chúng trong cửa sổ ngữ cảnh vẫn còn nhiều hạn chế. Đây là điểm mà LoCoMo đơn thuần không thể phát hiện, nhưng RAGAS có thể bóc tách và định lượng rõ ràng.

VI. KẾT QUẢ THỰC TIỄN

VII. KẾT LUẬN

TÀI LIỆU

- [1] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting for retrieval-augmented large language models," 2023.
- [2] C.-H. Tan, D. Bollegala *et al.*, "Memochat: Tuning llms to use memos for consistent long-range open-domain conversation," *arXiv preprint arXiv:2308.08239*, 2023.
- [3] Y. Zhong and Y. Gao, "Hingemem: Boundary guided long-term memory with query adaptive retrieval for scalable dialogues," *arXiv preprint arXiv:2604.06845*, 2026.
- [4] A. Maharana *et al.*, "Locomo: A large-scale multi-modal corpus for long-term memory and reasoning in conversational agents," *arXiv preprint*, 2024. [Online]. Available: <https://github.com/snap-research/locomo>
- [5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024. [Online]. Available: <https://arxiv.org/abs/2309.15217>

- [6] Y. G. M. Y. Y. S. Bernal Jiménez Gutiérrez, Yiheng Shu, "Hipporag: Neurobiologically inspired long-term memory for large language models," *arXiv:2405.14831*, 2025.
- [7] J. H. Sizhe Zhou, "Emem: A simple yet strong baseline for long-term conversational memory of llm agents," *arXiv:2511.17208*, 2026.